

Information–Theoretic Barriers for Collatz Proofs

Craig A. Feinstein

Machine-Checked Formalization in Isabelle/HOL

Abstract

In an earlier paper, *The Collatz $3n+1$ Conjecture is Unprovable* (2012), the author argued that any proof of the Collatz conjecture must encode arbitrarily large amounts of parity information, leading to the conclusion that no finite proof can exist. The present work formalises the mathematical framework underlying that argument. Under explicit assumptions concerning the representation and preservation of computational trace information, we prove that no finite proof certificate can establish the required convergence property. The result provides a machine-checked information-theoretic barrier theorem inspired by the earlier argument.

Contents

1	Collatz map and parity vectors	3
2	Affine formula characterisation	5
3	Two-adic invariance	9
4	Every parity vector is realisable	11
5	Proof system setup	15
6	The unprovability theorem	15

Provenance

This formalisation is inspired by:

C. A. Feinstein, “*The Collatz $3n + 1$ Conjecture is Unprovable*”,
arXiv:math/0312309; *Global Journal of Science Frontier Research*,
Mathematics and Decision Sciences, Volume 12, Issue 8 (2012), 13–15.

The present development formalises the information-theoretic ideas underlying the author’s earlier paper *The Collatz $3n + 1$ Conjecture is Unprovable*. The assumptions required for that argument are made explicit and formalised within Isabelle/HOL, yielding a machine-checked theorem showing that no finite proof certificate can exist within the corresponding class of proof systems.

These assumptions are motivated by structural properties of the Collatz map, including the realisability of arbitrary parity vectors, the injective dependence of affine parameters on parity traces, and the essential role of parity information in determining the dynamics.

The author of this formalisation received assistance from two AI systems — ChatGPT (OpenAI) and Claude (Anthropic). Their assistance consisted of drafting and refining explanatory text, improving the readability of the introduction and comments, and helping diagnose or structure Isabelle/HOL proof scripts.

Main goal

The present development investigates the consequences of requiring a proof system to represent Collatz trace information explicitly. The resulting theorem establishes an information-theoretic barrier for a broad class of trace-based proof methods based on explicit representations of parity information.

High-level strategy

The argument formalised here isolates the following phenomenon:

1. Certain proof strategies must expose or encode complete computational traces of the Collatz iteration (parity vectors).
2. The Collatz map realises parity vectors of arbitrary length, including information-theoretically incompressible ones.
3. Under explicit assumptions governing how proofs represent information, finite proof certificates cannot accommodate arbitrarily long incompressible traces.

The structure of the argument is inspired by Chaitin-style incompressibility methods, but is applied to the representation of computational traces within proof certificates rather than to the computation of specific strings.

Structure of the formalisation

1. **Collatz map and parity vectors** We define the simplified Collatz function T and formalise parity vectors as computational traces of the iteration.
2. **Affine formula characterisation** We show that $T^{(k)}(n)$ admits an affine representation $(3^s \cdot n + c)/2^k$, and prove that the parameters (k, s, c) are uniquely determined by the parity vector.
3. **Two-adic invariance** We establish a key invariance property: adding 2^k to a starting value does not affect the first k parity bits. This property underlies the realisability construction.
4. **Every parity vector is realisable** We prove that every finite binary string occurs as the parity vector of some starting value. This shows that arbitrarily complex computational traces are inherent to the Collatz dynamics.
5. **Proof system setup** We introduce an abstract model of proof certificates based on bitstrings, substring containment, and information-theoretic compressibility.
6. **A trace-based barrier theorem** Under explicit assumptions on trace containment, incompressibility preservation, and unbounded verification, we derive a formal barrier result: no finite proof exists within this class of proof systems.

```
theory Collatz_Information_Barriers
  imports Main
begin
```

1 Collatz map and parity vectors

The Collatz function

We define the simplified Collatz function T by

$$T(n) = \begin{cases} n/2 & \text{if } n \text{ is even,} \\ (3n + 1)/2 & \text{if } n \text{ is odd.} \end{cases}$$

```
definition T :: "nat ⇒ nat" where
  "T n = (if even n then n div 2 else (3*n + 1) div 2)"
```

```
abbreviation Tpow :: "nat ⇒ nat ⇒ nat"
  where "Tpow k n ≡ (T ^^ k) n"
```

The Collatz conjecture (unbounded version)

The standard formulation of the Collatz conjecture is:

$$\forall n. \exists k. T^{(k)}(n) = 1.$$

An equivalent unbounded formulation is:

$$\forall n \forall L. \exists k \geq L. T^{(k)}(n) = 1.$$

These formulations are equivalent. Once a number reaches 1, the Collatz iteration enters the cycle

$$1 \rightarrow 2 \rightarrow 1 \rightarrow 2 \rightarrow \dots$$

and therefore reaches 1 infinitely many times. Consequently, for any threshold L , there exists some $k \geq L$ such that $T^{(k)}(n) = 1$.

The parity vector as computational trace

The parity vector *parity_vec* n k records the sequence of even and odd values encountered during the first k iterations starting from n :

$$\text{parity_vec } n \ k = [\text{odd}(n), \text{odd}(T(n)), \text{odd}(T^{(2)}(n)), \dots, \text{odd}(T^{(k-1)}(n))].$$

This parity vector represents the *computational trace*: the sequence of branching decisions taken during the iteration of the Collatz map.

Example

For $n = 27$ we have:

$$T(27) = 41, \quad T^{(2)}(27) = 62, \quad T^{(3)}(27) = 31.$$

Thus:

$$\begin{aligned} \text{parity_vec } 27 \ 4 &= [\text{odd}(T^{(0)}(27)), \text{odd}(T^{(1)}(27)), \text{odd}(T^{(2)}(27)), \text{odd}(T^{(3)}(27))] \\ &= [\text{odd}(27), \text{odd}(41), \text{odd}(62), \text{odd}(31)] \\ &= [\text{True}, \text{True}, \text{False}, \text{True}]. \end{aligned}$$

```
definition parity_vec :: "nat  $\Rightarrow$  nat  $\Rightarrow$  bool list"
  where "parity_vec n k = map ( $\lambda i$ . odd (Tpow i n)) [0.. $k$ ]"

lemma length_parity_vec[simp]: "length (parity_vec n k) = k"
  by (simp add: parity_vec_def)
```

2 Affine formula characterisation

After k iterations of the Collatz map T , the result admits an affine representation:

$$T^k(n) = \frac{3^s \cdot n + c}{2^k}.$$

Here:

- k is the number of iterations.
- s counts the number of odd values in the parity vector, that is, the number of steps of the form $(3n + 1)/2$.
- c is a constant determined by the specific parity sequence.

Intuition

Each odd step multiplies the current value by 3, adds 1, and then divides by 2, while each even step simply divides by 2. After k steps, the cumulative effect is multiplication by 3^s , division by 2^k , and the addition of an accumulated constant c .

Uniqueness

The parameters (k, s, c) are uniquely determined by the parity vector. In particular, the parity vector encodes all computational information needed to reconstruct the affine formula for $T^{(k)}(n)$.

```
fun params :: "nat ⇒ bool list ⇒ nat × nat" where
  "params i [] = (0, 0)" |
  "params i (b # bs) =
    (let (c,s) = params (Suc i) bs in
     if b then (3*c + 2^i, Suc s) else (c, s))"
```

```
definition params0 :: "bool list ⇒ nat × nat" where
  "params0 x = params 0 x"
```

```
definition formula_of :: "bool list ⇒ nat × nat × nat" where
  "formula_of x = (length x, snd (params0 x), fst (params0 x))"
```

Injectivity

Different parity vectors produce different affine formula parameters. In particular, distinct parity sequences yield distinct triples (k, s, c) in the representation

$$T^k(n) = \frac{3^s \cdot n + c}{2^k}.$$

```
lemma pow2_dvd_fst_params_Suc:
```

```

"2 ^ Suc i dvd fst (params (Suc i) zs)"
proof (induction zs arbitrary: i)
  case Nil
  show ?case by simp
next
  case (Cons b bs)
  obtain c s where P: "params (Suc (Suc i)) bs = (c, s)"
  by (cases "params (Suc (Suc i)) bs") auto
  from Cons.IH[of "Suc i"] P have IH: "2 ^ Suc (Suc i) dvd c" by simp
  then obtain t where c_rep: "c = 2 ^ Suc (Suc i) * t"
  by (auto simp: dvd_def)
  have div_c: "2 ^ Suc i dvd c"
  proof (unfold dvd_def, intro exI)
    show "c = 2 ^ Suc i * (2 * t)"
    by (simp add: c_rep)
  qed
  with P show ?case
  by (cases b) simp_all
qed

lemma mod_drop_left_multiple_nat:
  fixes m a r :: nat
  shows "(m * a + r) mod m = r mod m"
  by (simp add: mod_add_left_eq mod_mult_left_eq)

lemma params_injective_len:
  assumes "length xs = length ys" and "params i xs = params i ys"
  shows "xs = ys"
  using assms
proof (induction xs arbitrary: ys i)
  case Nil
  then show ?case by (cases ys) auto
next
  case (Cons a xs)
  from Cons.prem1 obtain b ys' where [simp]: "ys = b # ys'"
  by (cases ys) auto

  obtain c s where Pxs: "params (Suc i) xs = (c,s)"
  by (cases "params (Suc i) xs") auto
  obtain c' s' where Pys: "params (Suc i) ys' = (c',s')"
  by (cases "params (Suc i) ys'") auto

  have Eq:
    "(if a then (3*c + 2^i, Suc s) else (c,s)) =
     (if b then (3*c' + 2^i, Suc s') else (c',s'))"
  using Cons.prem2 Pxs Pys by simp

  have len_tails: "length xs = length ys'" using Cons.prem1 by simp

  show ?case

```

```

proof -
  have "a = b"
  proof (rule ccontr)
    assume "a ≠ b"
    then consider (TF) "a" "¬ b" | (FT) "¬ a" "b" by auto
    then show False
  proof cases
    case TF
    from pow2_dvdfst_params_Suc[of i xs] Pxs
    have div_c: "2 ^ Suc i dvd c" by simp
    from pow2_dvdfst_params_Suc[of i ys'] Pys
    have div_c': "2 ^ Suc i dvd c'" by simp
    let ?M = "2 ^ Suc i"

    obtain t where c_rep: "c = ?M * t"
      using div_c by (auto simp: dvd_def)
    obtain t' where c'_rep: "c' = ?M * t'"
      using div_c' by (auto simp: dvd_def)

    have Lmod: "(3 * c + 2 ^ i) mod ?M = 2 ^ i"
    proof -
      have "(3 * c + 2 ^ i) mod ?M
        = ((?M * (3 * t)) + 2 ^ i) mod ?M" by (simp add: c_rep algebra_simps)
      also have "... = (2 ^ i) mod ?M"
        by (meson mod_drop_left_multiple_nat)
      also have "... = 2 ^ i" by simp
      finally show ?thesis .
    qed

    have Rmod: "c' mod ?M = 0" by (simp add: c'_rep)

    from Eq TF have "(3*c + 2^i, Suc s) = (c', s')" by simp
    hence "(3*c + 2^i) = c'" by simp
    hence "(3*c + 2^i) mod ?M = c' mod ?M" by simp
    hence "2 ^ i = 0" using Lmod Rmod by simp
    thus False using power_eq_0_iff by fastforce
  next
    case FT
    from pow2_dvdfst_params_Suc[of i ys'] Pys
    have div_c': "2 ^ Suc i dvd c'" by simp
    from pow2_dvdfst_params_Suc[of i xs] Pxs
    have div_c: "2 ^ Suc i dvd c" by simp
    let ?M = "2 ^ Suc i"

    obtain t' where c'_rep: "c' = ?M * t'"
      using div_c' by (auto simp: dvd_def)
    obtain t where c_rep: "c = ?M * t"
      using div_c by (auto simp: dvd_def)

    have Rmod: "(3 * c' + 2 ^ i) mod ?M = 2 ^ i"

```

```

proof -
  have "(3 * c' + 2 ^ i) mod ?M
    = ((?M * (3 * t')) + 2 ^ i) mod ?M" by (simp add: c'rep algebra_simps)
  also have "... = (2 ^ i) mod ?M"
    using mod_drop_left_multiple_nat by blast
  also have "... = 2 ^ i" by simp
  finally show ?thesis .
qed

have Lmod: "c mod ?M = 0" by (simp add: c_rep)

from Eq FT have "(c, s) = (3*c' + 2^i, Suc s')" by simp
hence "c = 3*c' + 2^i" by simp
hence "c mod ?M = (3*c' + 2^i) mod ?M" by simp
hence "0 = 2 ^ i" using Lmod Rmod by simp
thus False by (metis not_exp_less_eq_0_int verit_comp_simplify1(2))
qed
qed

then have bits_eq: "a = b" by simp
from Eq bits_eq have "c = c' ^ s = s'" by (cases a) auto
hence "xs = ys'"
  using Cons.IH[OF len_tails] Pxs Pys by metis
with bits_eq show ?thesis by simp
qed
qed

lemma formula_determines_parity_on_len:
  assumes "length x = k" "length y = k" "formula_of x = formula_of y"
  shows "x = y"
proof -
  from assms(3)
  have "snd (params0 x) = snd (params0 y)" "fst (params0 x) = fst (params0
y)"
    by (auto simp: formula_of_def)
  hence "params0 x = params0 y"
    by (cases "params0 x"; cases "params0 y"; simp)
  thus ?thesis
    using assms(1,2) params_injective_len[of x y 0]
    by (simp add: params0_def)
qed

```


3 Two-adic invariance

Key lemma

Adding $q \cdot 2^k$ to m preserves the first k parity bits:

$$\text{parity_vec}(m + q \cdot 2^k, k) = \text{parity_vec}(m, k).$$

Intuition

The offset $q \cdot 2^k$ is beyond the resolution of the first k steps. After one application of the Collatz map T , the offset becomes $q \cdot 2^{k-1}$ if m is even, or $3q \cdot 2^{k-1}$ if m is odd, and therefore remains a multiple of 2^{k-1} . This behaviour continues inductively through subsequent iterations.

Consequence

This invariance property enables realisability. One can tune a number to have any desired parity sequence by adding suitable multiples of 2^k , without affecting parity bits that have already been fixed.

```

lemma T_funpow_commute: "T ((T ^^ j) n) = (T ^^ j) (T n)"
proof (induction j)
  case 0
  show ?case by simp
next
  case (Suc j)
  have "T ((T ^^ Suc j) n) = T (T ((T ^^ j) n))" by simp
  also have "... = T ((T ^^ j) (T n))" by (simp add: Suc.IH)
  also have "... = (T ^^ Suc j) (T n)" by simp
  finally show ?case .
qed

lemma parity_vec_Suc:
  "parity_vec n (Suc k) = odd n # parity_vec (T n) k"
proof (rule nth_equalityI)
  show "length (parity_vec n (Suc k)) = length (odd n # parity_vec (T n) k)"
    by simp
next
  fix i assume iLt: "i < length (parity_vec n (Suc k))"
  then have iSk: "i < Suc k" by (simp add: parity_vec_def)
  consider (Z) "i = 0" | (S) j where "i = Suc j" "j < k"
    using iSk by (cases i) auto
  then show "parity_vec n (Suc k) ! i = (odd n # parity_vec (T n) k) ! i"
  proof cases
    case Z
    show ?thesis
      using Z by (simp add: parity_vec_def del: upt_Suc)
  next
    case (S j)

```

```

    show ?thesis
      using S
      by (simp add: parity_vec_def T_funpow_commute del: upt_Suc)
  qed
qed

lemma parity_vec_add_pow2_invariant:
  fixes m q k :: nat
  shows "parity_vec (m + q * 2 ^ k) k = parity_vec m k"
proof (induction k arbitrary: m q)
  case 0
  show ?case by (simp add: parity_vec_def)
next
  case (Suc k)
  let ?Δ = "q * 2 ^ Suc k"

  have head: "odd (m + ?Δ) = odd m"
    by simp

  have tail: "parity_vec (T (m + ?Δ)) k = parity_vec (T m) k"
  proof (cases "even m")
    case True
    then obtain t where m2: "m = 2*t" by (elim evenE)
    have "(m + ?Δ) div 2 = t + q * 2 ^ k"
      by (simp add: m2)
    hence "T (m + ?Δ) = t + q * 2 ^ k"
      using True by (simp add: T_def)
    moreover have "T m = t"
      using True m2 by (simp add: T_def)
    ultimately show ?thesis
      using Suc.IH[of t q] by simp
  next
    case False
    then obtain t where m2: "m = 2*t + 1" by (elim oddE)
    have "(3*(m + ?Δ) + 1) div 2
      = (3*m + 1) div 2 + (3*q) * 2 ^ k"
    proof -
      have "3*(m + ?Δ) + 1 = (3*m + 1) + (3*q) * 2 ^ Suc k"
        by simp
      moreover have "even (3*m + 1)"
        using False m2 by simp
      ultimately show ?thesis by simp
    qed
    hence "T (m + ?Δ) = T m + (3*q) * 2 ^ k"
      using False by (simp add: T_def)
    thus ?thesis
      using Suc.IH[of "T m" "3*q"] by simp
  qed

  have step1: "parity_vec (m + ?Δ) (Suc k)

```

```

      = odd (m + ?Δ) # parity_vec (T (m + ?Δ)) k"
    by (simp add: parity_vec_Suc)
  also have step2: "... = odd m # parity_vec (T m) k"
    using head tail by blast
  also have step3: "... = parity_vec m (Suc k)"
    by (simp add: parity_vec_Suc)
  finally show ?case .
qed

```

4 Every parity vector is realisable

Realisability

For any finite binary string x , there exists a natural number n whose parity sequence agrees with x :

$$\forall x. \exists n. \text{parity_vec } n (\text{length } x) = x.$$

This means that every possible computational trace of the Collatz map actually occurs for some starting number. In particular, parity traces of arbitrary length, including incompressible traces, are realised.

Helper lemmas for modular arithmetic

The following lemmas establish basic modular-arithmetic facts that are used in the realisability construction.

```

lemma power_mod_nat:
  fixes a m n :: nat
  shows "(a ^ n) mod m = ((a mod m) ^ n) mod m"
proof (induction n)
  case 0
  show ?case by simp
next
  case (Suc n)
  have "(a ^ Suc n) mod m = (a * a ^ n) mod m" by simp
  also have "... = (((a mod m) * a ^ n) mod m)"
    by (simp add: mod_mult_left_eq)
  also have "... = (((a mod m) * ((a ^ n) mod m)) mod m)"
    by (simp add: mod_mult_right_eq)
  also have "... = (((a mod m) * (((a mod m) ^ n) mod m)) mod m)"
    by (simp add: Suc.IH)
  also have "... = (((a mod m) * ((a mod m) ^ n)) mod m)"
    by (simp add: mod_mult_right_eq)
  also have "... = ((a mod m) ^ Suc n) mod m" by simp
  finally show ?case .
qed

```

```

lemma power_mult_nat:
  fixes a :: nat
  shows "a ^ (m * n) = (a ^ m) ^ n"
proof (induction n)
  case 0
  show ?case by simp
next
  case (Suc n)
  have "a ^ (m * Suc n) = a ^ (m*n + m)" by (simp add: add.commute)
  also have "... = a ^ (m*n) * a ^ m" by (simp add: power_add)
  also have "... = (a ^ m) ^ n * a ^ m" by (simp add: Suc.IH)
  also have "... = (a ^ m) ^ Suc n" by simp
  finally show ?case .
qed

lemma pow4_mod3: "(4::nat) ^ m mod 3 = 1"
  by (simp add: power_mod_nat)

lemma pow2_mod3_even:
  assumes "even 1"
  shows "(2 :: nat) ^ 1 mod 3 = 1"
proof -
  obtain m where L: "1 = 2*m" using assms by (erule evenE)
  have "2 ^ 1 mod 3 = (2 ^ (2*m)) mod 3" by (simp add: L)
  also have "... = ((2::nat) ^ 2) ^ m mod 3" by (simp add: power_mult_nat)
  also have "... = (4 ^ m) mod 3" by simp
  also have "... = 1" by (rule pow4_mod3)
  finally show ?thesis .
qed

lemma pow2_mod3_odd:
  assumes "odd 1"
  shows "(2 :: nat) ^ 1 mod 3 = 2"
proof -
  obtain m where L: "1 = Suc (2*m)" using assms by (metis Suc_eq_plus1 oddE)
  have "2 ^ 1 mod 3 = (2 * 2 ^ (2*m)) mod 3" by (simp add: L)
  also have "... = (2 * ((2::nat) ^ 2) ^ m) mod 3"
    by (simp add: power_mult_nat)
  also have "... = (2 * ((4 ^ m) mod 3)) mod 3"
    by (simp add: mod_mult_right_eq)
  also have "... = (2 * 1) mod 3" by (simp add: pow4_mod3)
  also have "... = 2" by simp
  finally show ?thesis .
qed

lemma pow2_mod3:
  fixes l :: nat
  shows "(2 :: nat) ^ 1 mod 3 = (if even 1 then 1 else 2)"
  by (cases "even 1") (simp add: pow2_mod3_even, simp add: pow2_mod3_odd)

```

```

lemma choose_t_even_mod3:
  fixes m0 l :: nat
  assumes "even l"
  shows " $\exists t \leq 2. (m0 + t * (2 :: nat) ^ 1) \bmod 3 = 2$ "
proof -
  define t where "t = (2 - (m0 mod 3)) mod 3"
  have t_le2: "t ≤ 2" by (simp add: t_def)

  have "(m0 + t * 2 ^ 1) mod 3
    = (m0 mod 3 + t * (2 ^ 1 mod 3)) mod 3"
    by (metis mod_add_cong mod_mod_trivial mod_mult_right_eq)
  also have "... = (m0 mod 3 + t) mod 3"
    using assms by (simp add: pow2_mod3)
  also have "... = (m0 mod 3 + ((2 - (m0 mod 3)) mod 3)) mod 3"
    by (simp add: t_def)
  also have "... = 2"
    by (cases "m0 mod 3") simp_all
  finally have targ: "(m0 + t * 2 ^ 1) mod 3 = 2" .
  from t_le2 targ show ?thesis by blast
qed

lemma choose_t_odd_mod3:
  fixes m0 l :: nat
  assumes "odd l"
  shows " $\exists t \leq 2. (m0 + t * (2 :: nat) ^ 1) \bmod 3 = 2$ "
proof -
  define t where "t = (2 * (2 - (m0 mod 3))) mod 3"
  have t_le2: "t ≤ 2" by (simp add: t_def)
  have "(m0 + t * 2 ^ 1) mod 3
    = (m0 mod 3 + t * (2 ^ 1 mod 3)) mod 3"
    by (metis mod_add_cong mod_mod_trivial mod_mult_right_eq)
  also have "... = (m0 mod 3 + ((2 * (2 - (m0 mod 3))) mod 3) * 2) mod 3"
    by (simp add: t_def pow2_mod3 assms)
  also have "... = (m0 mod 3 + (4 * (2 - (m0 mod 3))) mod 3) mod 3"
    using mod_mult_right_eq by (metis (no_types, lifting)
      distrib_right mod_add_eq mod_add_left_eq mult_2_right numeral_Bit0_eq_double)
  also have "... = (m0 mod 3 + ((4 mod 3) *
    ((2 - (m0 mod 3)) mod 3)) mod 3) mod 3"
    using mod_mult_left_eq by (metis mod_mult_right_eq)
  also have "... = (m0 mod 3 + ((2 - (m0 mod 3)) mod 3)) mod 3" by simp
  also have "... = 2" by (cases "m0 mod 3") simp_all
  finally have targ: "(m0 + t * 2 ^ 1) mod 3 = 2" .
  show ?thesis using t_le2 targ by blast
qed

lemma parity_vector_realizable:
  fixes x :: "bool list"
  shows " $\exists n. \text{parity\_vec } n (\text{length } x) = x$ "
proof (induction x)
  case Nil

```

```

show ?case by (metis length_0_conv length_parity_vec)
next
case (Cons b bs)
obtain m0 where IH: "parity_vec m0 (length bs) = bs"
  using Cons.IH by blast
show ?case
proof (cases b)
  case False
  let ?n = "2*m0"
  have "parity_vec ?n (length (b # bs))
    = odd ?n # parity_vec (T ?n) (length bs)"
    by (simp add: parity_vec_Suc)
  also have "... = False # bs" by (simp add: T_def IH)
  also have "... = b # bs" by (simp add: False)
  finally show ?thesis by (intro exI[of _ ?n])
next
case True
let ?l = "length bs"
obtain t where t_le2: " $t \leq 2$ " and targ: " $(m0 + t * 2 ^ ?l) \bmod 3 = 2$ "
  by (cases "even ?l")
  (use choose_t_even_mod3[of ?l m0] choose_t_odd_mod3[of ?l m0] in auto)
let ?m = "m0 + t * 2 ^ ?l"
have tail_preserved: "parity_vec ?m ?l = bs"
  using IH parity_vec_add_pow2_invariant[of m0 t ?l] by simp
define q where "q = ?m div 3"
have m_eq: "?m = 3*q + 2"
proof -
  have "?m = 3 * (?m div 3) + (?m mod 3)" by (simp add: div_mult_mod_eq)
  also have "... = 3*q + 2" by (simp add: q_def targ)
  finally show ?thesis .
qed
let ?n = "(2 * ?m - 1) div 3"
have head: "odd ?n"
proof -
  have "?n = (2 * (3*q + 2) - 1) div 3" by (simp add: m_eq)
  also have "... = (6*q + 3) div 3" by simp
  also have "... = 2*q + 1" by simp
  finally show ?thesis by simp
qed
have tail_step: "parity_vec (T ?n) ?l = bs"
  using tail_preserved head by (simp add: m_eq T_def)
have "parity_vec ?n (length (b # bs)) = odd ?n # parity_vec (T ?n) ?l"
  by (simp add: parity_vec_Suc)
also have "... = True # bs" using head tail_step by simp
also have "... = b # bs" by (simp add: True)
finally show ?thesis by (intro exI[of _ ?n])
qed
qed

```

5 Proof system setup

```
type_synonym bit = bool
type_synonym bitstring = "bit list"
```

Substring containment

We introduce a substring-containment relation as a *concrete and explicit* model of proofs that store computational trace information as literal data. Formally, a bitstring p contains a bitstring s if s occurs verbatim as a contiguous substring of p , i.e. if there exist bitstrings u and v such that $p = u@s@v$.

Remark. The choice of substring containment is deliberately strong. It provides a simple, syntactic notion of explicit information storage that is easy to reason about formally and avoids ambiguity about how information is represented inside a proof certificate. In the main barrier theorem, we exploit only the elementary consequence that storing a string of length m requires at least m bits of space. Stronger notions of information encoding (for example, distributed or implicit encodings) are intentionally left outside the scope of this development.

```
definition contains :: "bitstring  $\Rightarrow$  bitstring  $\Rightarrow$  bool"
  where "contains p s  $\longleftrightarrow$  ( $\exists$  u v. p = u @ s @ v)"
```

```
lemma contains_len_bound: "contains p s ==> length s <= length p"
  by (auto simp: contains_def)
```

6 The unprovability theorem

```
lemma finite_bitstrings_of_len:
  "finite {s::bitstring. length s = m}"
proof -
  have fin_aux:
    "finite {s::bool list. set s <= (UNIV::bool set)  $\wedge$  length s = m}"
    by (rule finite_lists_length_eq) simp
  have "{s::bitstring. length s = m}
    = {s. set s <= (UNIV::bool set)  $\wedge$  length s = m}"
    by auto
  then show ?thesis using fin_aux by (simp only:)
qed
```

```
lemma many_strings_of_length:
  "card {s::bitstring. length s = m} = 2 ^ m"
proof (induction m)
  case 0
  have "{s::bitstring. length s = 0} = {[]}" by auto
  thus ?case by simp
next
```

```

case (Suc m)
have "{s. length s = Suc m} =
  (%b. True # b) ` {s. length s = m} Un (%b. False # b) ` {s. length s =
m}"
  by (auto simp: length_Suc_conv)
moreover have "(\b. True # b) ` {s. length s = m}
  Int (\b. False # b) ` {s. length s = m} = {}"
  by auto
moreover have "inj_on (\b. True # b) {s. length s = m}"
  by (auto simp: inj_on_def)
moreover have "inj_on (\b. False # b) {s. length s = m}"
  by (auto simp: inj_on_def)
ultimately show ?case
  using Suc.IH card_Un_disjoint card_image
  by (smt (verit) Suc_1 Suc_pred card.infinite diff_add_zero mult_2 nat.discI

      plus_1_eq_Suc power_Suc0_right power_add power_eq_0_iff zero_less_one)
qed

definition compressible :: "bitstring  $\Rightarrow$  (bitstring  $\Rightarrow$  bitstring)  $\Rightarrow$  bool"
where
  "compressible s enc  $\equiv$  length (enc s) < length s"

definition incompressible_by :: "bitstring  $\Rightarrow$  (bitstring  $\Rightarrow$  bitstring)  $\Rightarrow$  bool"
where
  "incompressible_by s enc  $\equiv$   $\neg$ compressible s enc"

lemma sum_pow2_lt: "sum (%i. (2::nat) ^ i) {...<m} = 2 ^ m - 1"
  by (induction m) simp_all

lemma finite_bitstrings_le_len:
  "finite {s::bitstring. length s  $\leq$  m}"
proof (induction m)
  case 0 show ?case by (simp add: finite_bitstrings_of_len)
next
  case (Suc m)
  have "{s::bitstring. length s <= Suc m}
    = {s. length s <= m} Un {s. length s = Suc m}" by auto
  thus ?case using Suc.IH finite_bitstrings_of_len by (simp add: finite_UnI)
qed

lemma card_bitstrings_le_len:
  "card {s::bitstring. length s <= m} = sum (%i. 2 ^ i) {...m}"
proof -
  let ?S = "\i. {s::bitstring. length s = i}"
  have union_eq: "{s::bitstring. length s <= m} = Union ((%i. ?S i) ` {...m})"
    by auto
  have fin_index: "finite {...m::nat set}" by simp
  have fin_each: "!!i. i <= m ==> finite (?S i)"
    by (simp add: finite_bitstrings_of_len)

```



```

have disj: "!!i j. i <= m ==> j <= m ==> i ~= j ==> ?S i Int ?S j = {}"
  by auto
have "card (Union ((%i. ?S i) ` {..m})) = sum (%i. card (?S i)) {..m}"
  by (rule card_UN_disjoint) (use fin_index fin_each disj in auto)
also have "... = sum (%i. 2 ^ i) {..m}"
  by (simp add: many_strings_of_length)
finally show ?thesis
  by (simp add: union_eq)
qed

```

The pigeonhole argument

Theorem. For any injective encoding, there exists an incompressible bitstring.

```

theorem incompressible_strings_exist_for_enc:
  fixes enc :: "bitstring => bitstring"
  assumes "inj enc"
  shows "∃ s. length s = m ∧ incompressible_by s enc"
proof (cases m)
  case 0
  have "length ([]::bitstring) = 0 ∧ 0 ≤ length (enc [])"
    by simp
  then show ?thesis using 0 incompressible_by_def compressible_def by auto
next
  case (Suc r)
  let ?S = "{t::bitstring. length t = Suc r}"
  let ?T = "{u::bitstring. length u ≤ r}"
  have finS: "finite ?S" by (simp add: finite_bitstrings_of_len)
  have finT: "finite ?T" by (simp add: finite_bitstrings_le_len)
  have Sm: "card ?S = 2 ^ Suc r"
    by (simp add: many_strings_of_length)
  have Tm: "card ?T = sum (%i. (2::nat) ^ i) {..r}"
    by (simp add: card_bitstrings_le_len)
  also have "... = sum (%i. (2::nat) ^ i) {..

```

```

then obtain  $t$  where  $tS$ : " $t : ?S$ " and  $len$ : " $\sim length (enc\ t) \leq r$ " by blast

hence " $length (enc\ t) \geq Suc\ r$ " by simp
moreover from  $tS$  have " $length\ t = Suc\ r$ " by auto
ultimately show  $?thesis$ 
  using  $Suc\ incompressible\_by\_def\ compressible\_def$  by auto
qed

```

Proof system locale

This locale models a class of proof-certificate encodings used by trace-based approaches to Collatz. We introduce three explicit assumptions capturing a class of trace-based proof systems.

1. **Containment.** Proofs must encode parity information. This is not derived from first principles, but is assumed based on structural properties of the Collatz function, namely realisability, injectivity of the affine formula, and opposite monotonicity.
2. **Incompressibility preservation.** The encoding cannot compress truly incompressible information. This is a basic information-theoretic constraint.
3. **Unbounded verification.** A proof of the Collatz conjecture must work for arbitrarily large iteration counts. This follows from the equivalence of the standard and unbounded formulations.

The resulting unprovability statement is therefore conditional. If every valid proof must encode parity vectors, and if such encodings preserve information-theoretic incompressibility, then no finite proof of the Collatz $3n + 1$ conjecture can exist.

Justification of the containment assumption

The assumption

$$\text{is_proof_of_convergence}(p, k, n) \implies p \text{ contains } \text{enc_parity}(\text{parity_vec } n\ k)$$

formalises the idea that any proof asserting that n converges to 1 within k steps must, at least implicitly, encode the parity vector of the first k Collatz iterates of n . This assumption is justified by three structural properties that are specific to the Collatz function.

1. **Realisability of all parity vectors** For the Collatz map, every finite bit-string occurs as a parity vector. As a result, any proof of universal convergence

must account for all possible parity patterns; none can be excluded a priori. By contrast, consider the function

$$T_1(n) = \begin{cases} n/2 & \text{if } n \text{ is even,} \\ n+1 & \text{if } n \text{ is odd.} \end{cases}$$

This map is not realisable in the above sense: after an odd step, the next value is always even, so the parity pattern [True, True] never occurs. Proofs for T_1 therefore do not need to consider arbitrary parity strings.

2. Opposite monotonicity In the Collatz map, even steps always decrease the value, while odd steps always increase it. Because the size may increase or decrease depending on the parity, the behaviour of the iteration cannot be understood without knowing which branch was taken at each step. By contrast, consider the function

$$T_2(n) = \begin{cases} n/2 & \text{if } n \text{ is even,} \\ (n+1)/2 & \text{if } n \text{ is odd.} \end{cases}$$

Both branches decrease for $n > 1$, so convergence can be proved using a global monotone invariant, without referring to individual parity choices.

3. Injectivity of the affine formula For the Collatz map, we have the identity

$$T^k(n) = \frac{3^s \cdot n + c}{2^k},$$

where the parameters (k, s, c) depend on the parity vector. As shown earlier, these parameters uniquely determine the parity vector. Consequently, any argument that relies on the affine formula implicitly determines the full parity sequence.

Why these properties force parity encoding

The Collatz function simultaneously satisfies the following three properties:

1. Every bitstring is realisable as a parity vector.
2. The behaviour of the iteration is unpredictable without branch information.
3. The affine parameters encode the parity vector injectively.

Because realisability forces a proof to account for all parity patterns, opposite monotonicity prevents parity information from being bypassed, and injectivity shows that affine data and parity data are equivalent, these properties provide strong motivation for the containment assumption that proofs effectively encode

the relevant parity vector. Since these properties fail for many superficially similar functions, the containment assumption is not arbitrary; it reflects a genuine structural feature of the actual Collatz dynamics.

```

locale Collatz_Trace_Barrier =
  fixes enc_parity :: "bool list  $\Rightarrow$  bitstring"
    and is_proof_of_convergence :: "bitstring  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  bool"
    and is_collatz_proof :: "bitstring  $\Rightarrow$  bool"

  assumes proof_contains_parity:
    "is_proof_of_convergence p k n
      $\Rightarrow$  contains p (enc_parity (parity_vec n k))"

  assumes encoding_preserves_incompressibility:
    "[| incompressible_by s enc_parity; length s = m; take m t = s |]
      $\Rightarrow$  length (enc_parity t)  $\geq$  m"

  assumes collatz_proof_unbounded:
    "is_collatz_proof p  $\Rightarrow$  ALL n L. EX k $\geq$ L. is_proof_of_convergence p k n"
begin

lemma incompressible_parity_encodings_exist:
  assumes "inj enc_parity"
  shows " $\exists$  s. length s = m  $\wedge$  incompressible_by s enc_parity"
  using incompressible_strings_exist_for_enc[OF assms] .

lemma parity_vec_prefix:
  assumes "m  $\leq$  k"
  shows "take m (parity_vec n k) = parity_vec n m"
  using assms by (simp add: parity_vec_def take_map)

```

Interpretation of the main theorem

Under Assumptions (1)–(3), no finite proof exists within this class of proof systems. The theorem is conditional on these assumptions, which are motivated by structural properties of the Collatz map.

```

theorem no_finite_collatz_proof:
  assumes "inj enc_parity"
  assumes "is_collatz_proof p"
  shows False
proof -
  define L where "L = length p"

  obtain s where
    s_len: "length s = Suc L" and
    s_incomp: "incompressible_by s enc_parity"
  using incompressible_parity_encodings_exist[OF assms(1), of "Suc L"] by
  blast

```

```

obtain n where pv_eq: "parity_vec n (Suc L) = s"
  using parity_vector_realizable[of s] s_len by auto

have "∃ k ≥ Suc L. is_proof_of_convergence p k n"
  using assms(2) collatz_proof_unbounded by blast

then obtain k where
  k_ge: "k ≥ Suc L" and
  k_proof: "is_proof_of_convergence p k n"
  by blast

have "take (Suc L) (parity_vec n k) = parity_vec n (Suc L)"
  using k_ge parity_vec_prefix by auto
also have "... = s" using pv_eq by simp
finally have prefix: "take (Suc L) (parity_vec n k) = s" .

have enc_large: "length (enc_parity (parity_vec n k)) ≥ Suc L"
  using encoding_preserves_incompressibility[OF s_incomp s_len prefix] by
blast

have contains_fact: "contains p (enc_parity (parity_vec n k))"
  using proof_contains_parity[OF k_proof] .

have len_le: "length (enc_parity (parity_vec n k)) ≤ length p"
  using contains_len_bound[OF contains_fact] .

have "Suc L ≤ length p"
  using enc_large len_le by linarith

thus False
  by (simp add: L_def)
qed

corollary no_collatz_proof_in_this_system:
  assumes "inj enc_parity"
  shows "¬ (∃ p. is_collatz_proof p)"
  using no_finite_collatz_proof[OF assms] by blast

end
end

```